

Xử lý ảnh & Thị giác máy tính

Chương 2. Các phép biến đổi

Mục tiêu bài học

- ☐ Biết và hiểu được phép toán tích chập
- ☐ Biết được các ứng dụng phép tích chập trong xử lý ảnh
- ☐ Biết và hiểu được phép biến đổi Fourier

Nội dung bài học

- ❑ Phép toán tích chập (Convolution)
- ❑ Ứng dụng phép toán tích chập trong xử lý ảnh
- ❑ Phép biến đổi Fourier (Fourier Transform)

2.1 Tích chập

- ❑ Tích chập là một phép toán quan trọng trong xử lý tín hiệu và xử lý ảnh.
- ❑ Tích chập thực hiện đối với 2 tín hiệu (hàm số) **f** và **g**, cho ra 1 tín hiệu (hàm số) thứ 3 **h**.

$$(f * g)(n) = h(n)$$

Tích chập

- ❑ Trong thực tế, tín hiệu và bộ lọc (kernel) có độ dài hữu hạn.
- ❑ Giả sử tín hiệu đầu vào f có độ dài N và kernel g có độ dài K . Công thức tích chập 1 chiều là:

$$(f * g)(i) = \sum_{j=0}^{K-1} f(i-j) \cdot g(j)$$

Với $0 \leq i \leq N-K$

Cần phải **lật** kernel g trước khi tích chập

Ví dụ:



$$f = \begin{bmatrix} 10 & 50 & 60 & 10 & 20 & 40 & 30 \end{bmatrix}$$

f: tín hiệu đầu vào

$$g = \begin{bmatrix} 1/3 & 1/3 & 1/3 \end{bmatrix}$$

g: bộ lọc(kernel)

h: ?

Quá trình tính đầu ra **h** có thể được thực hiện như sau:

1. Lật bộ lọc **g**
2. Đặt bộ lọc **g** vào vị trí tương ứng với **f** (phần tử đầu tiên của **f**).
3. Nhân từng phần tử tương ứng của **g** và **f** rồi cộng các kết quả lại để được phần tử tương ứng của **h**.
4. Trượt bộ lọc **g** một bước sang bên phải. Nếu phần tử cuối cùng của bộ lọc không vượt ra ngoài phần tử cuối cùng của tín hiệu, quay lại **Bước 2**. Ngược lại, dừng các bước tính toán.

Ví dụ:



$$f = \begin{bmatrix} 10 & 50 & 60 & 10 & 20 & 40 & 30 \end{bmatrix}$$

f: tín hiệu đầu vào

$$g = \begin{bmatrix} 1/3 & 1/3 & 1/3 \end{bmatrix}$$

g: bộ lọc(kernel)

Ta có:

$$(f*g)[0] = (10 \times 1/3) + (50 \times 1/3) + (60 \times 1/3) = 40$$

$$(f*g)[1] = (50 \times 1/3) + (60 \times 1/3) + (10 \times 1/3) = 40$$

$$(f*g)[2] = (60 \times 1/3) + (10 \times 1/3) + (20 \times 1/3) = 30$$

$$(f*g)[3] = (10 \times 1/3) + (20 \times 1/3) + (40 \times 1/3) = 23.333$$

$$(f*g)[4] = (20 \times 1/3) + (40 \times 1/3) + (30 \times 1/3) = 30$$

h: output

40	40	30	23.333	30
-----------	-----------	-----------	---------------	-----------

Ví dụ

Sử dụng hàm `convolve` để kiểm chứng kết quả tích chập:

```
import scipy as sp  
output = sp.convolve(f,g,'valid')
```

$$f = \begin{bmatrix} 10 & 50 & 60 & 10 & 20 & 40 & 30 \end{bmatrix}$$

$$g = \begin{bmatrix} 1/3 & 1/3 & 1/3 \end{bmatrix}$$

Output: `[40. 40. 30. 23.333 30. ]`

Tích chập

- ❑ Tích chập 2 chiều (*)
- ❑ Tương quan chéo - Cross-correlation ($\otimes$)

Tích chập

□ Tích chập 2 chiều (*)

$$G[i, j] = \sum_{u=0}^{K-1} \sum_{v=0}^{L-1} F[i - u, j - v] \cdot h[u, v]$$

F: Đầu vào (ma trận 2 chiều)

h: Bộ lọc (kernel, thường nhỏ hơn F, cần **lật Flip** trước khi tích chập)

G: Đầu ra

K, L: Kích thước của bộ lọc

Ví dụ Tích chập 2 chiều

F			
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

*

h		
1	1	0
1	0	1
0	1	0

$$G[i, j] = \sum_{u=0}^{K-1} \sum_{v=0}^{L-1} F[i - u, j - v] \cdot h[u, v]$$

Bước 1: “Lật” (flip) kernel **h** theo chiều ngang và dọc trước khi chập



0	1	0
1	0	1
0	1	1

```
import scipy as sp
output = sp.convolve2d(f,h,'valid')
```

Ví dụ Tích chập 2 chiều

F			
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

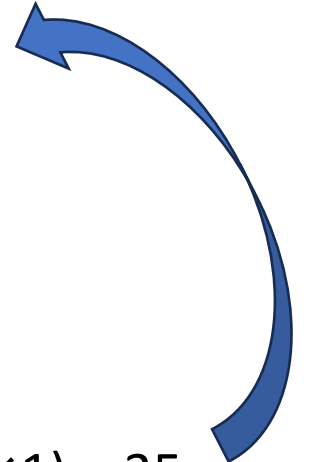
*

h		
0	1	0
1	0	1
0	1	1

$$G[i, j] = \sum_{u=0}^{K-1} \sum_{v=0}^{L-1} F[i - u, j - v] \cdot h[u, v]$$

=

35	40
50	60



Bước 2: Trượt kernel, nhân và tính tổng các phần tử

$$G[0,0] = (1 \times 0) + (2 \times 1) + (3 \times 0) + (5 \times 1) + (6 \times 0) + (7 \times 1) + (9 \times 0) + (10 \times 1) + (11 \times 1) = 35$$

$$G[0,1] = (2 \times 0) + (3 \times 1) + (4 \times 0) + (6 \times 1) + (7 \times 0) + (8 \times 1) + (10 \times 0) + (11 \times 1) + (12 \times 1) = 40$$

$$G[1,0] = (5 \times 0) + (6 \times 1) + (7 \times 0) + (9 \times 1) + (10 \times 0) + (11 \times 1) + (13 \times 0) + (14 \times 1) + (15 \times 1) = 50$$

$$G[1,1] = (6 \times 0) + (7 \times 1) + (8 \times 0) + (10 \times 1) + (11 \times 0) + (12 \times 1) + (14 \times 0) + (15 \times 1) + (16 \times 1) = 60$$

Tích chập

□ Tương quan chéo - Cross-correlation ($\otimes$)

$$G[i, j] = \sum_{u=0}^{K-1} \sum_{v=0}^{L-1} F[i + u, j + v] \cdot h[u, v]$$

F: Đầu vào (ma trận 2 chiều)

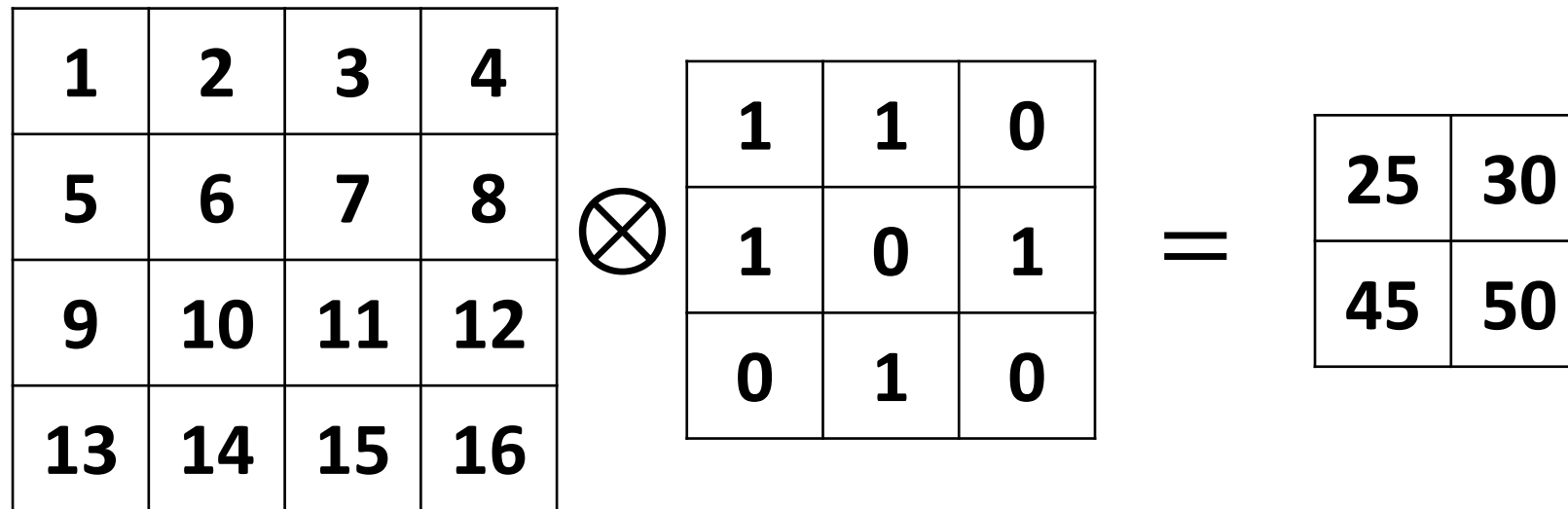
h: Bộ lọc (kernel, thường nhỏ hơn F, **không lật Flip**)

G: Đầu ra

K, L: Kích thước của bộ lọc

Ví dụ Cross-correlation

$$G[i, j] = \sum_{u=0}^{K-1} \sum_{v=0}^{L-1} F[i + u, j + v] \cdot h[u, v]$$



Sử dụng lệnh `import scipy as sp`
`output = sp.correlate2d(f,h,'valid')`

Tích chập

□ Tính giao hoán

$$f(x, y) * g(x, y) = g(x, y) * f(x, y)$$

□ Tính kết hợp

$$[f(x, y) * g(x, y)] * h(x, y) = f(x, y) * [g(x, y) * h(x, y)]$$

□ Tính phân phối

$$f(x, y) * [g(x, y) + h(x, y)] = [f(x, y) * g(x, y)] + [f(x, y) * h(x, y)]$$

Tích chập trong xử lý ảnh

Input * Bộ lọc = Output

kernel: Ma trận nhỏ kích thước $n \times n$, dùng để biến đổi giá trị pixel ban đầu của ma trận ảnh **input**.

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

*

2	1	0
0	2	2
2	1	0

=

Padding

❑ Không có padding

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12	12	17
10	17	19
9	6	14

❑ Có padding (zero)

0 ₂	0 ₀	0 ₁	0	0	0	0
0 ₁	2 ₀	2 ₀	3	3	3	0
0 ₀	0 ₁	1 ₁	3	0	3	0
0	2	3	0	1	3	0
0	3	3	2	1	2	0
0	3	3	0	2	3	0
0	0	0	0	0	0	0

1	6	5
7	10	9
7	10	8

Các mode của tích chập 1 chiều

- ❑ **valid** : Kích thước output **nhỏ hơn** kích thước input.
- ❑ **full** : Kích thước output **lớn hơn** kích thước input.
- ❑ **same** : Kích thước output **bằng** kích thước input.

Tính kích thước tín hiệu đầu ra:

Valid : $N - K + 1$

Full : $N + K - 1$

Same : N

Trong đó:

N: Kích thước tín hiệu đầu vào

K: Kích thước bộ lọc (kernel)

Các mode của tích chập 2 chiều

Tính kích thước tín hiệu đầu ra:

Valid : Chiều ngang = $N-K+1$, chiều dọc = $M-L+1$

Full : Chiều ngang = $N+K-1$, chiều dọc = $M+L-1$

Same : Chiều ngang = N , chiều dọc = M

Trong đó:

N, M : Kích thước tín hiệu đầu vào

K, L : Kích thước bộ lọc (kernel)

Các mode của hàm tích chập convolve



- ❑ Cách dùng 3 mode **full**, **same**, **valid** của hàm `convolve` và `convolve2d`
`from scipy.signal import sp`

Tích chập một chiều:

```
f = convolve(img, kernel, 'full')  
s = convolve(img, kernel, 'same')  
v = convolve(img, kernel, 'valid')
```

Tích chập hai chiều:

```
f = convolve2d(img, kernel, 'full')  
s = convolve2d(img, kernel, 'same')  
v = convolve2d(img, kernel, 'valid')
```

Tương quan chéo:

```
f = correlate2d(img, kernel, 'full')  
s = correlate2d(img, kernel, 'same')  
v = correlate2d(img, kernel, 'valid')
```

Ví dụ:



$$f = \begin{bmatrix} 10 & 50 & 60 & 10 & 20 & 40 & 30 \end{bmatrix}$$

f: tín hiệu đầu vào

$$g = \begin{bmatrix} 1/3 & 1/3 & 1/3 \end{bmatrix}$$

g: bộ lọc(kernel)

h: ?

Hãy tìm **h** (output) theo 2 mode "**full**" và "**same**"?

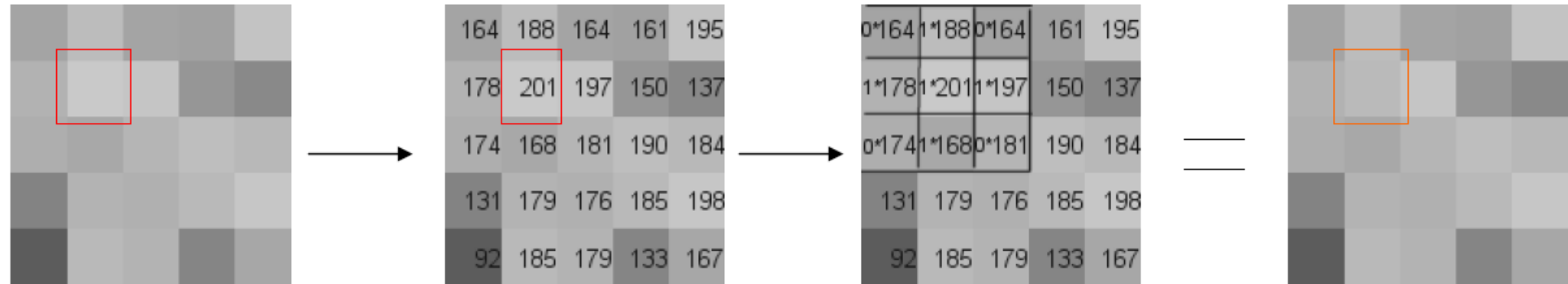


h:

Full: [3.333 20. 40. 40. 30. 23.333 30. 23.333 10.]

Same: [20. 40. 40. 30. 23.333 30. 23.333]

Tích chập trong xử lý ảnh



Original image

Image with color values placed over it

Image with 3x3 kernel placed over it

Output image

164	188	164
178	201	197
174	168	181



0	1	0
1	1	1
0	1	0

Divided by the sum of the kernel

$$932 \div 5 = \text{new pixel color}$$

Tích chập trong xử lý ảnh

Ma trận ảnh * Kernel (filter)



Original



0	0	0
0	1	0
0	0	0



Identical image

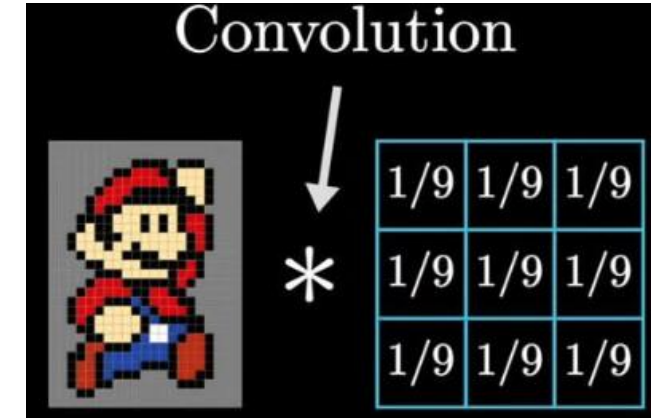
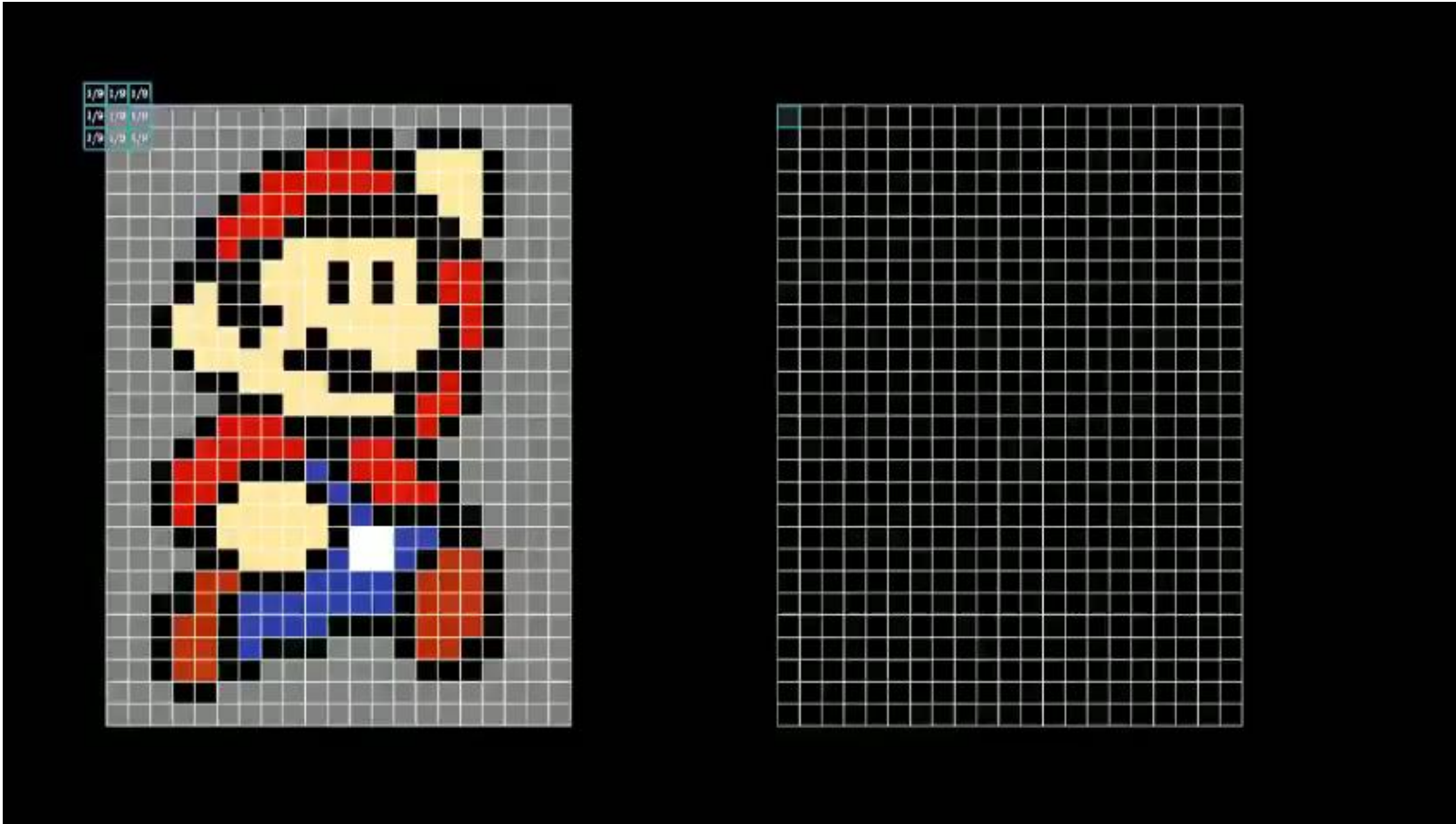
Lưu ý: Cần chuyển dữ liệu về dạng số thực float32

2.2 Ứng dụng tích chập trong xử lý ảnh

Một số ứng dụng:

- ☐ Làm mờ (blurring)
- ☐ Làm sắc nét (sharpen)
- ☐ Nhận biết vùng biên (edge detection)

Làm mờ (blur)



Original



Sharp



kernel

-1	-1	-1
-1	9	-1
-1	-1	-1

Intensified sharper image

cameramen.tif

Blur



Edge detection



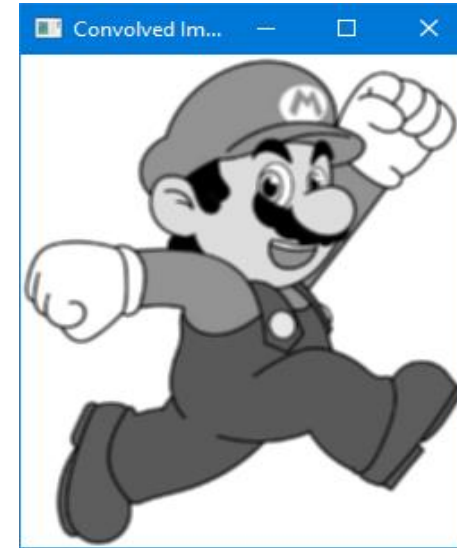
kernel

0	-1	0
-1	5	-1
0	-1	0

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \text{ kernel}$$



$$* \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$



$$* \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$



- ❑ Tích chập được áp dụng chủ yếu để cải thiện ảnh trên miền không gian, tức tính toán trực tiếp trên ma trận.
→ Từ ảnh ban đầu, dùng **một số bộ lọc** được thiết kế để **tích chập** với **ảnh gốc** theo yêu cầu sử dụng

- ❑ Trên miền tần số, cần phải có một công cụ để phân tích.
→ Biến đổi Fourier

Ôn tập

Các hàm xử lý ảnh

- ❑ `I = imread("logo_vaa.jpg")` # đọc file ảnh tên logo_vaa.jpg, lưu vào biến I
- ❑ `imshow("VAA", I)` # show hình ảnh của I
- ❑ `imwrite("logo_save .jpg", I)` # lưu ảnh I thành ảnh "logo_save .jpg"
- ❑ `cvtColor(I, cv2.COLOR_BGR2GRAY)` # chuyển ảnh màu I thành ảnh xám
- ❑ `calcHist(I, 0, None, [256], [0, 256])` # tính histogram của ảnh I

Lập trình trên Pycharm

- ❑ Tạo project, tạo file .py trong project
- ❑ Import thư viện (ví dụ: `import numpy as np`)
- ❑ Comment/uncomment trên PyCharm bằng `ctrl + /` và `ctrl + /`, hoặc sau dấu `#`

OpenCV



```
1. import cv2
2. import numpy as np
3.
4. # Đọc ảnh đầu vào (ảnh xám)
5. image = cv2.imread('mario.png', cv2.IMREAD_GRAYSCALE) # IMREAD_COLOR
6.
7. # Tạo bộ lọc kernel 3x3 (ví dụ: kernel làm mờ ảnh)
8. kernel = np.array([[1, 1, 1],
9.                    [1, 1, 1],
10.                   [1, 1, 1]], dtype=np.float32)
11.
12. # Chuẩn hóa kernel để tổng của các phần tử là 1 (cho kernel làm mờ)
13. kernel = kernel / 9.0
14.
15. # Áp dụng tích chập 2 chiều sử dụng hàm filter2D
16. output = cv2.filter2D(image, -1, kernel)
17.
18. # Hiển thị ảnh gốc và ảnh sau khi áp dụng tích chập
19. cv2.imshow('Original Image', image)
20. cv2.imshow('Convolved Image', output)
```

2.3 Biến đổi Fourier

Cải thiện ảnh trong miền tần số

- ❑ Khái niệm biến đổi Fourier

- ❑ Lọc trong miền tần số

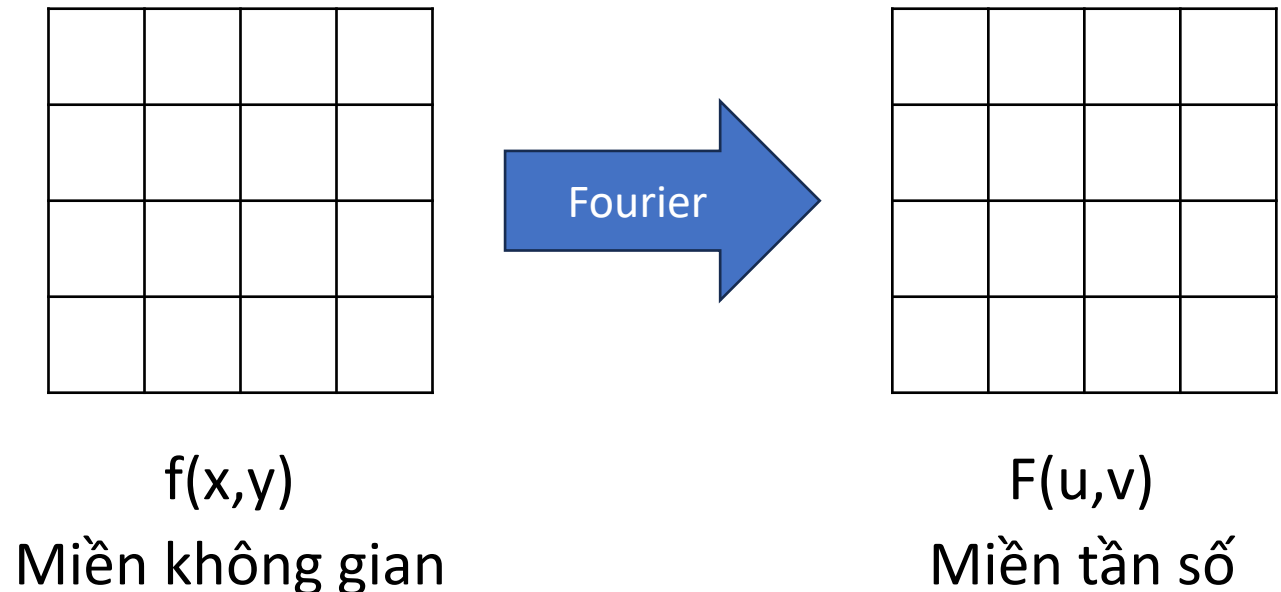
 - ❑ Lọc thông thấp

 - ❑ Lọc thông cao

 - ❑ Lý tưởng (ideal)

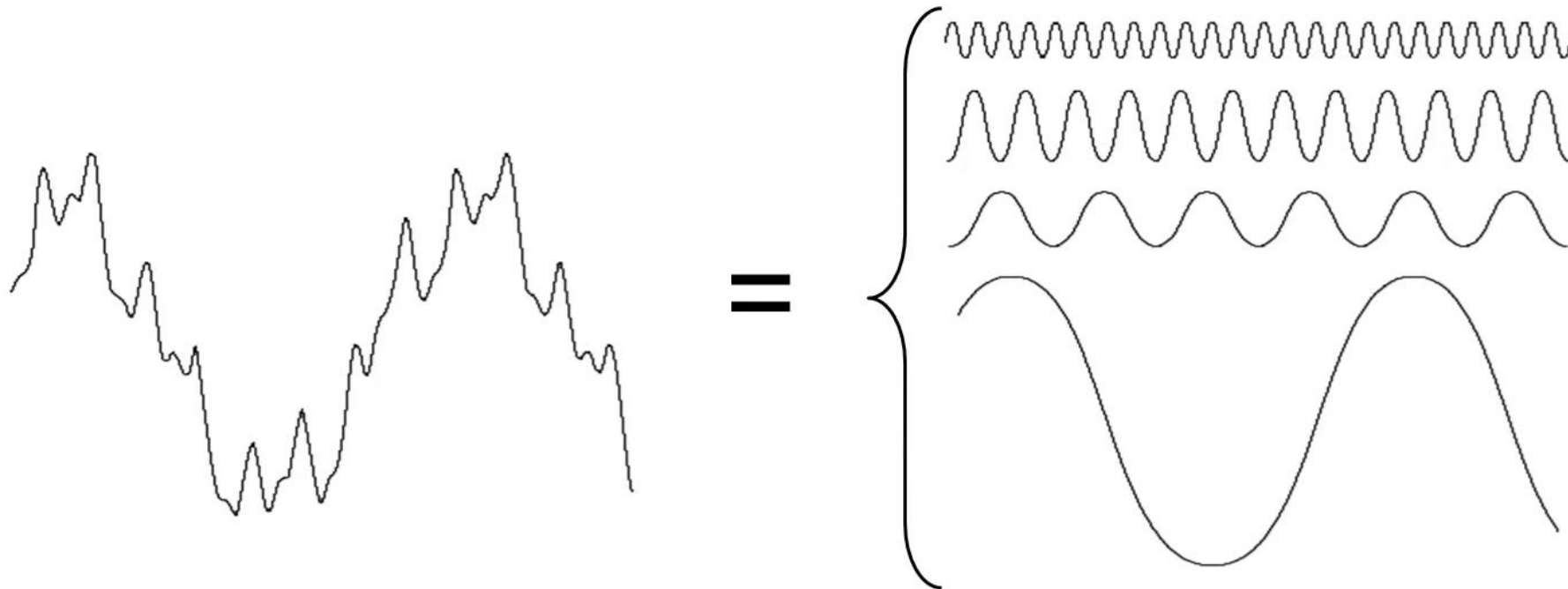
 - ❑ Gaussian

 - ❑ Butterworth



Biến đổi Fourier

- ❑ Một hàm bất kỳ lặp lại có tính chu kỳ có thể được biểu diễn bằng tổng những hàm **sine** và **cosine** ở các tần số khác nhau → là **chuỗi Fourier**



Số phức

Số phức được định nghĩa (tổng quát):

$$C = R + jI \quad (1)$$

Trong đó:

R là phần thực

I là phần ảo

Đơn vị ảo j , và $j^2 = -1$

Số phức

❑ Biểu diễn trong hệ tọa độ góc: $C = |C| \times (\cos\theta + j\sin\theta)$ (2)

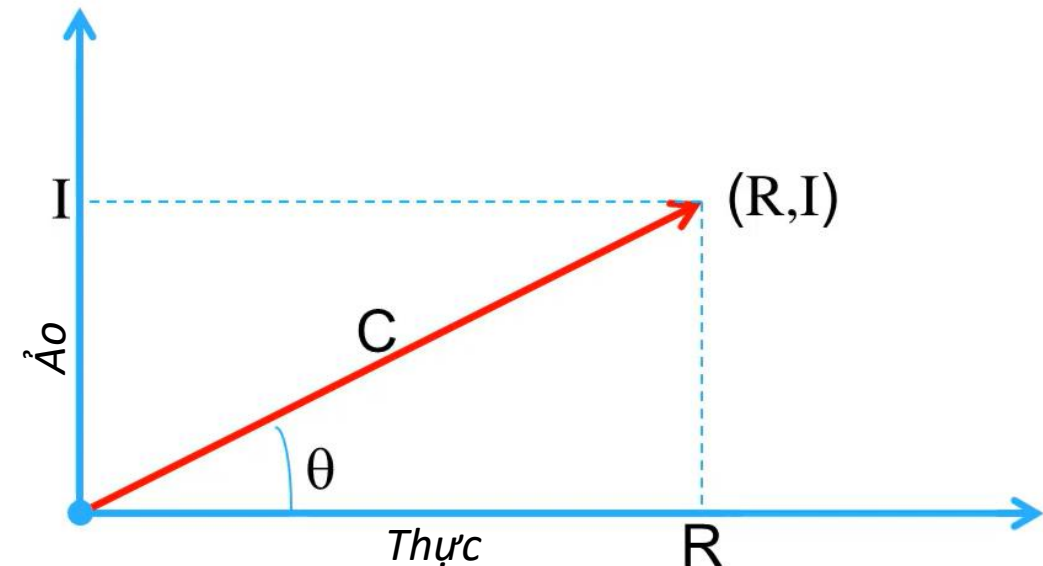
❑ $\theta = \arctan(\frac{I}{R})$: góc giữa vector C với trục thực (R)

❑ $|C| = \sqrt{R^2 + I^2}$: độ dài (lớn) của vector từ góc đến điểm (R,I) trên mặt phẳng phức

❑ Từ công thức Euler:

$$e^{j\theta} = \cos\theta + j\sin\theta$$

Có thể viết lại $C = |C|e^{j\theta}$ (3)



Biến đổi Fourier rời rạc

□ Biến đổi Fourier rời rạc (DFT) 1 chiều

$$F(u) = \sum_{x=0}^{M-1} f(x) e^{-j2\pi ux/M} \quad (1)$$

□ Công thức Euler

$$e^{j\theta} = \cos\theta + j\sin\theta$$

□ Thế công thức Euler vào (1)

$$F(u) = \sum_{x=0}^{M-1} f(x) \left[\frac{\cos 2\pi ux}{M} - j \frac{\sin 2\pi ux}{M} \right]$$

Trong đó:

- **F(u)**: là giá trị biến đổi Fourier tại chỉ số tần số k
- **f(x)**: là giá trị của tín hiệu tại mẫu thứ x
- **e^{-j2πux/M}**: là hàm mũ phức
- **M**: số lượng mẫu
- **u = 0, 1...M-1**: là chỉ số tần số rời rạc

Biên độ và phổ

□ Do hàm trong miền tần số là số phức, ta có thể viết lại trong **tọa độ góc**:

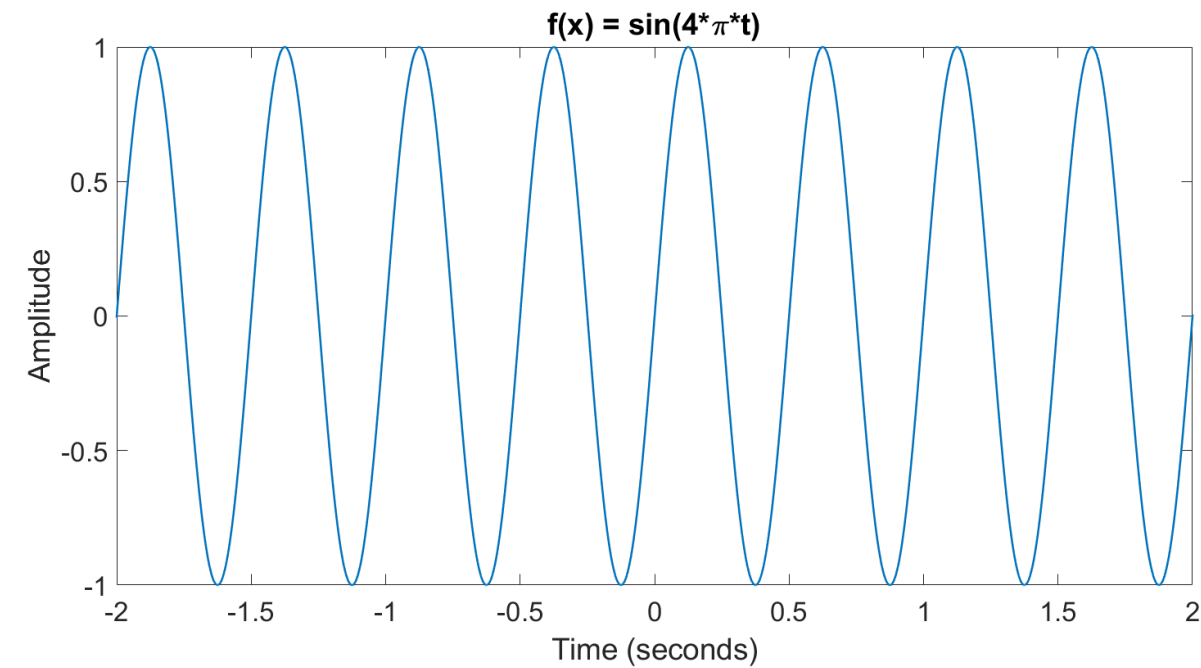
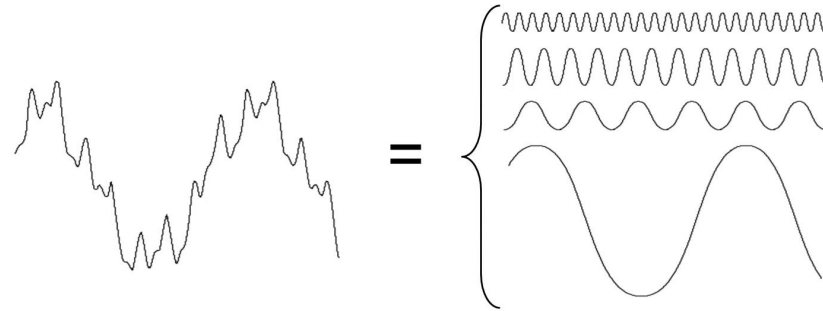
$$F(u) = |F(u)|e^{-j\phi(u)}$$

Với $|F(u)| = [R^2(u) + I^2(u)]^{1/2}$ được gọi là **biên độ**

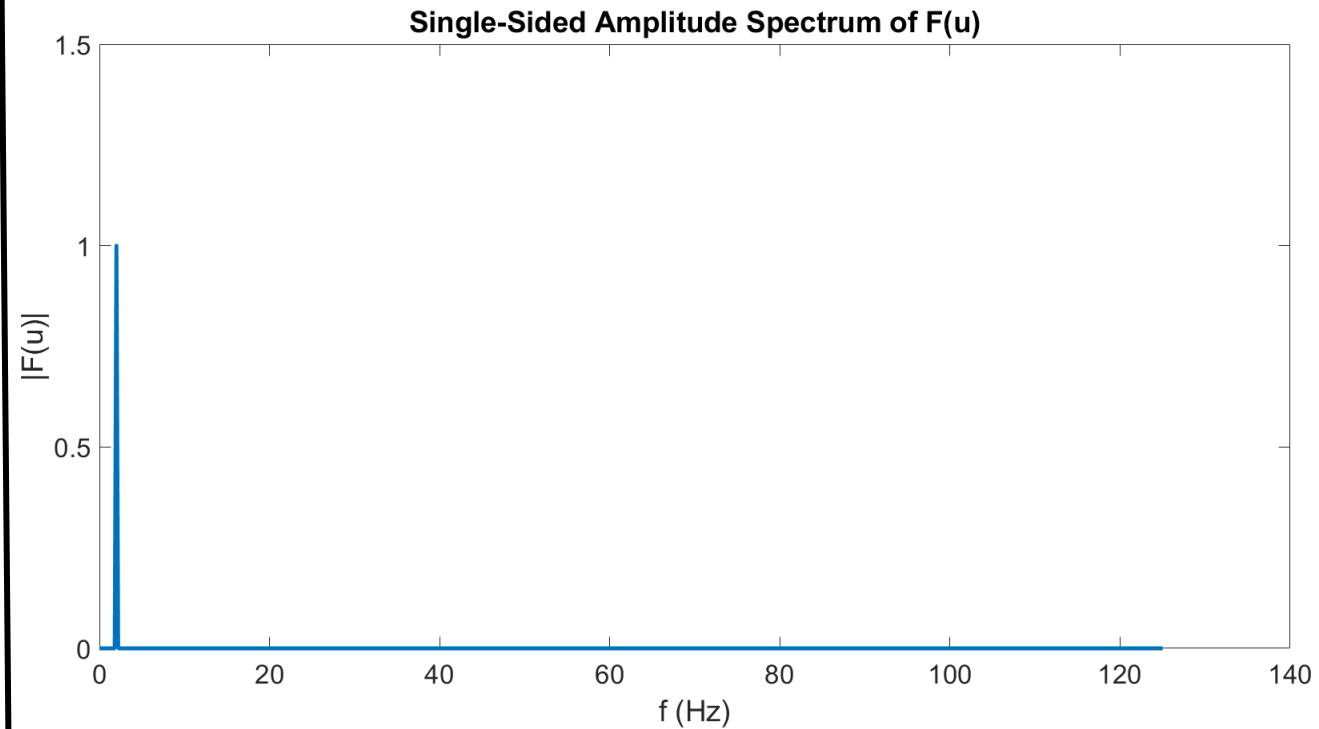
$\phi(u) = \arctan \left[\frac{I(u)}{R(u)} \right]$ được gọi là **phổ/góc pha**

$P(u) = |F(u)|^2$ là **năng lượng phổ**

Ví dụ



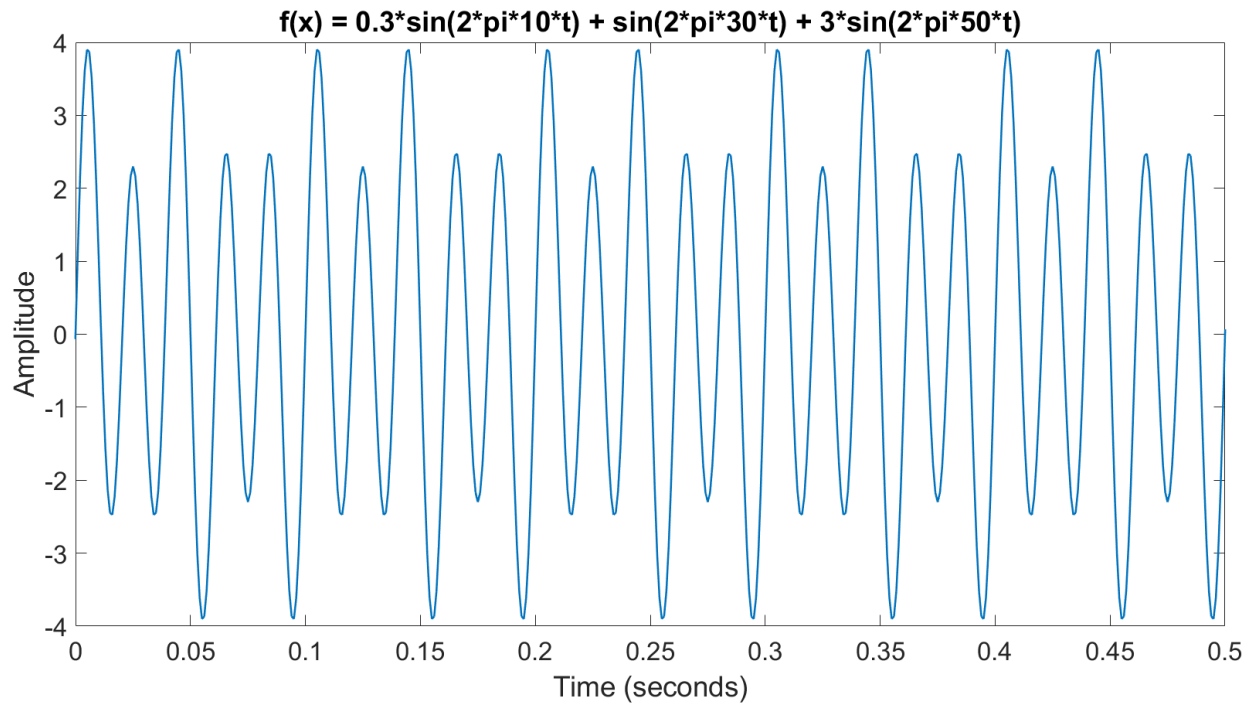
$$f(x) = \sin(4\pi t)$$



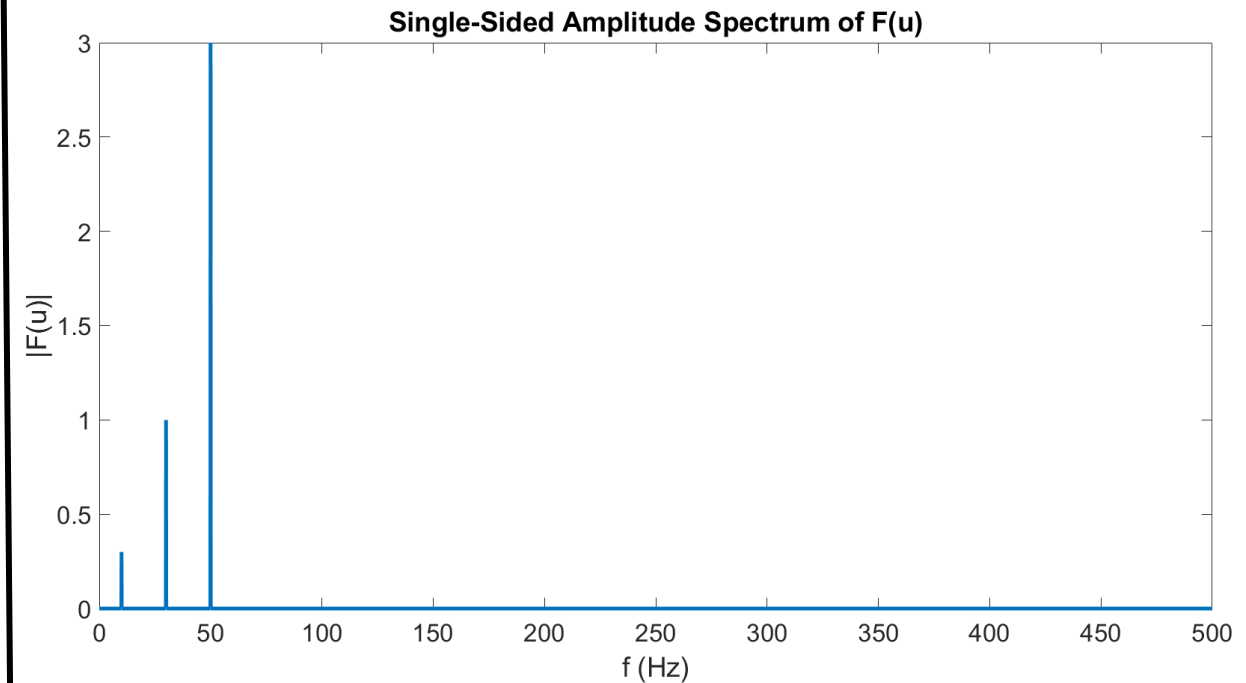
$$F(u)$$

Tín hiệu gốc $f(x)$ tập trung tại tần số $f = 2\text{Hz}$

Ví dụ



$$f(x) = 0.3 \sin(20\pi t) + \sin(60\pi t) + 3 \sin(100\pi t)$$

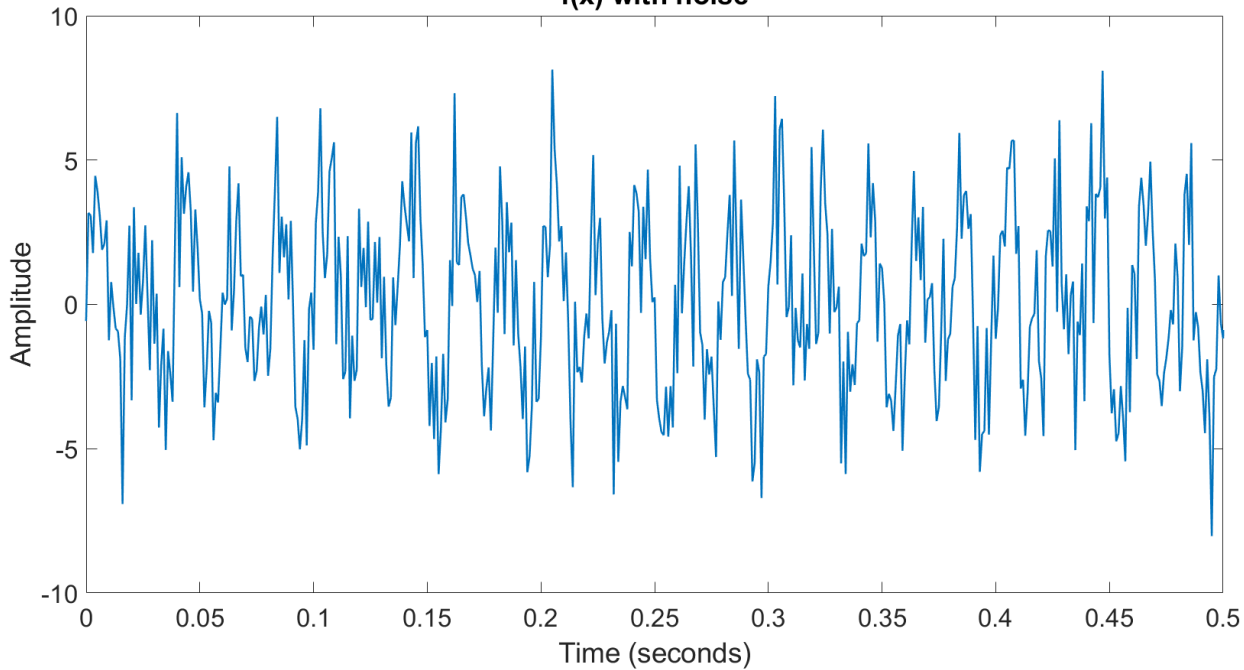


$F(u)$

Tín hiệu gốc $f(x)$ tập trung tại các tần số $f = 10\text{Hz}, 30\text{Hz}, 50\text{Hz}$

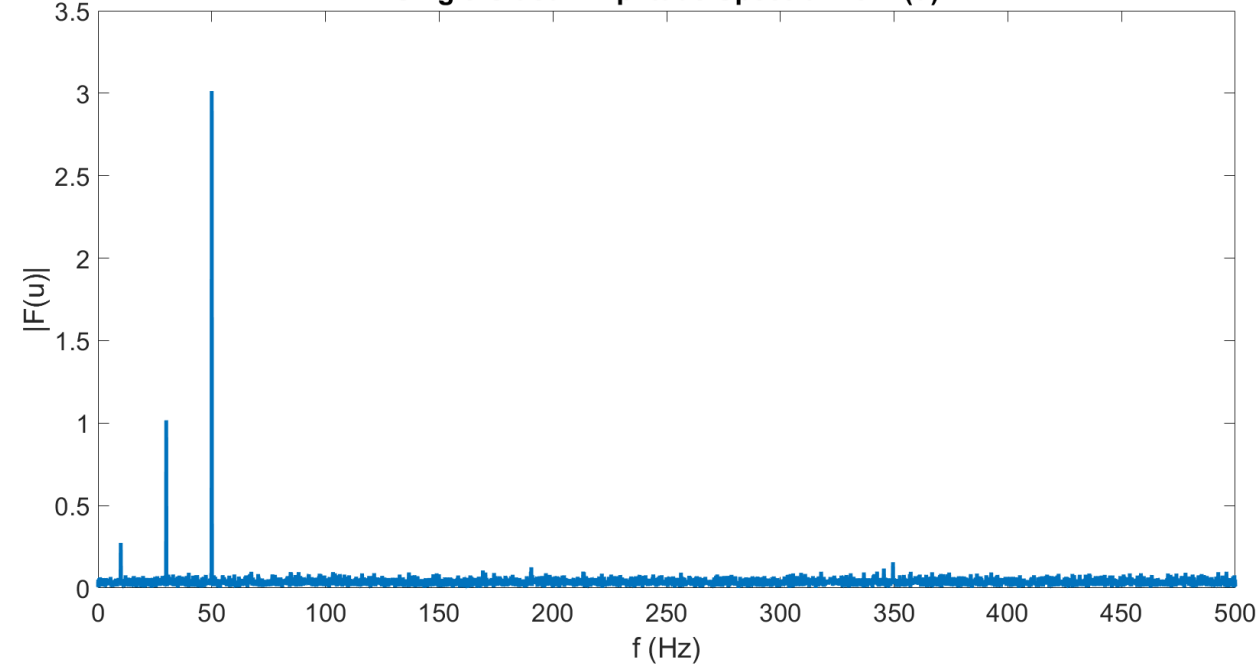
Ví dụ

f(x) with noise



$$f(x) = 0.3\sin(20\pi t) + \sin(60\pi t) + 3\sin(100\pi t) + \text{noise}$$

Single-Sided Amplitude Spectrum of F(u)



Tín hiệu gốc $f(x)$ tập trung tại các tần số $f = 10\text{Hz}, 30\text{Hz}, 50\text{Hz}$

Ứng dụng Fourier

☐ **Cải thiện ảnh trong miền tần số:**

- ☐ Làm mờ ảnh, loại bỏ chi tiết tần số cao (Low-pass filter)
- ☐ Làm sắc nét ảnh, loại bỏ tần số thấp giữ lại chi tiết (High-pass filter)
- ☐ Nén ảnh, phát hiện biên cạnh, khử nhiễu ...

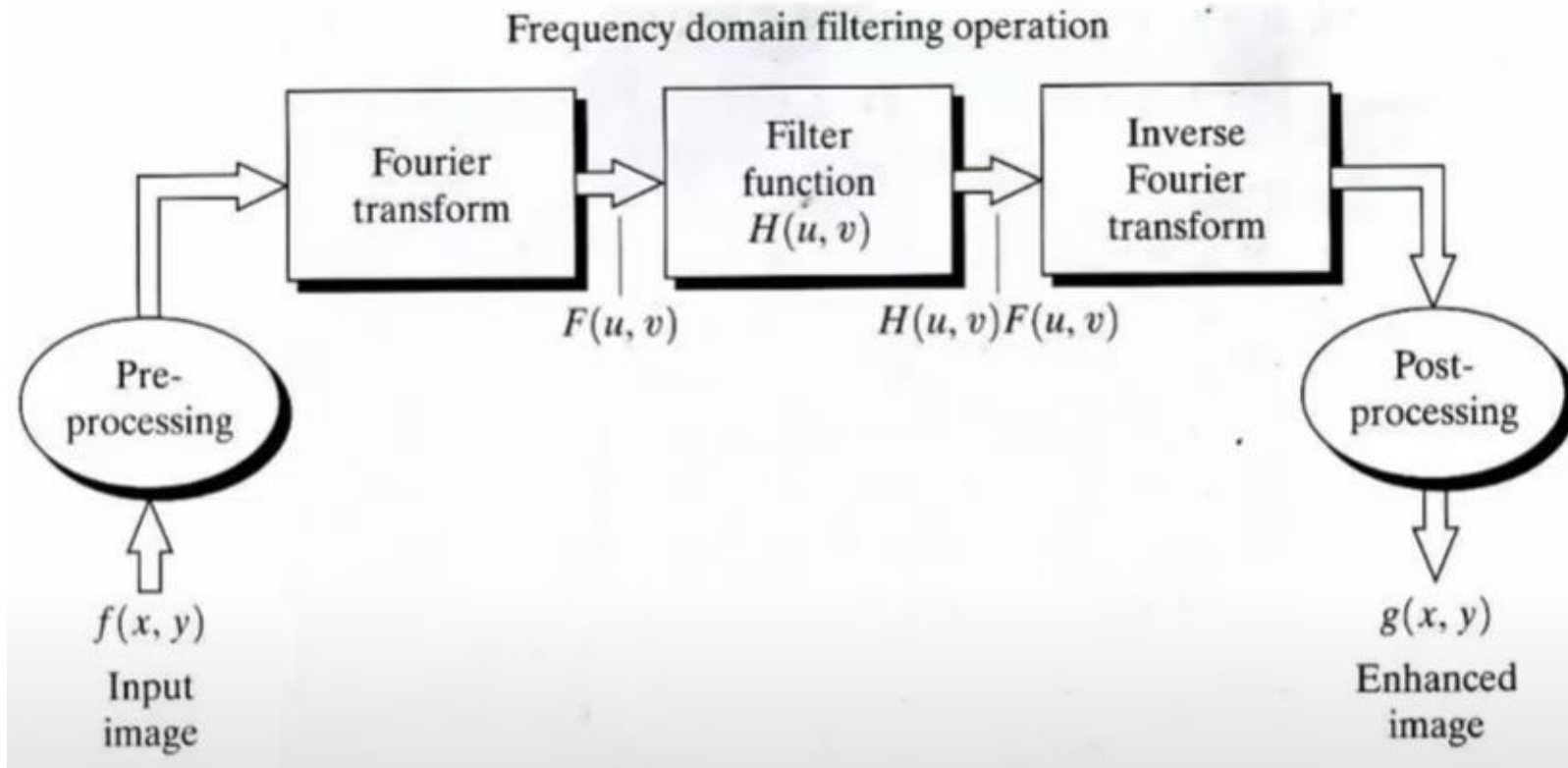
Code

```
# tạo hàm số  $x = \sin(4\pi t)$ , tần số lấy mẫu là 250
>> Fs = 250                                # tần số lấy mẫu là 250, T = 0.004
>> T = 1/Fs                                # T = 0.004
>> t = np.arange(0, 5, T)                  # khoảng thời gian 0 đến 5
>> x = np.sin(4 * np.pi * t)              # tại hàm  $\sin$ , tần số bằng 2Hz
#-----#
```

Sử dụng hàm `fft(x)` của thư viện `numpy (np)` để phân tích phổ của tín hiệu `x` được tạo ở trên.

Biến đổi Fourier trong xử lý ảnh

□ DFT trong xử lý ảnh là DFT 2 chiều



Ứng dụng:

- Lọc ảnh
- Nén ảnh
- Phát hiện cạnh biên

Biến đổi Fourier trong xử lý ảnh

□ Công thức biến đổi Fourier rời rạc 2 chiều (DFT)

$$G(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} g(x, y) e^{-j\frac{2\pi(ux)}{M}} e^{-j\frac{2\pi(vy)}{N}}$$

Trong đó:

- **$G(u, v)$** : là giá trị của phổ Fourier tại tần số u, v trong miền tần số (ảnh Fourier)
- **$g(x, y)$** : là giá trị cường độ ảnh tại tọa độ x, y trong miền không gian (ảnh gốc)
- $e^{-j\frac{2\pi(ux)}{M}} e^{-j\frac{2\pi(vy)}{N}}$: là hàm mũ phức biểu diễn các thành phần tần số
- **M, N** : lần lượt là số điểm ảnh theo trục x, y
- **$u=0, 1...M-1$** và **$v=0, 1...N-1$** : là các chỉ số tần số rời rạc tương ứng với x và y

Biến đổi Fourier trong xử lý ảnh

□ Công thức biến đổi Fourier rời rạc ngược 2 chiều (IDFT)

$$g(x, y) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} G(u, v) e^{j\frac{2\pi(ux)}{M}} e^{j\frac{2\pi(vy)}{N}}$$

Trong đó:

- **$G(u, v)$** : là giá trị của phổ Fourier tại tần số u, v trong miền tần số (ảnh Fourier)
- **$g(x, y)$** : là giá trị cường độ ảnh tại tọa độ x, y trong miền không gian (ảnh gốc)
- $e^{j\frac{2\pi(ux)}{M}} e^{-j\frac{2\pi(vy)}{N}}$: là hàm mũ phức ngược với dấu dương trong mũ
- **M, N** : lần lượt là số điểm ảnh theo trục x, y

Công thức biến đổi Fourier cho ảnh

□ Nếu $G(u,v)$ là ảnh Fourier của một ảnh $g(x,y)$ có M dòng, N cột:

$$G = U^T \times g \times V \quad (1)$$

Với U, V là các ma trận:

$$U(x, u) = e^{-\frac{j2\pi xu}{M}} ; x, u = 0:M-1$$

$$V(y, v) = e^{-\frac{j2\pi yv}{N}} ; y, v = 0:N-1$$

Công thức biến đổi Fourier cho ảnh

□ Nếu $U(u, x) = U(x, u) \rightarrow U = U^T$, (1) có thể viết lại:

$$G = U \times g \times V$$

□ Biến đổi Fourier ngược:

$$g = U^{-1} \times G \times V^{-1}$$

Ví dụ biến đổi Fourier

□ Cho một ảnh 4x4 $g(x,y)$. Tìm ảnh Fourier của ảnh g

$$g(x,y) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$U(x,u) = e^{-\frac{j2\pi xu}{M}}; x, u = 0 : M - 1$$

$$U = \begin{bmatrix} e^{\frac{-j \times 2 \times \pi \times 0 \times 0}{4}} & e^{\frac{-j \times 2 \times \pi \times 0 \times 1}{4}} & e^{\frac{-j \times 2 \times \pi \times 0 \times 2}{4}} & e^{\frac{-j \times 2 \times \pi \times 0 \times 3}{4}} \\ e^{\frac{-j \times 2 \times \pi \times 1 \times 0}{4}} & e^{\frac{-j \times 2 \times \pi \times 1 \times 1}{4}} & e^{\frac{-j \times 2 \times \pi \times 1 \times 2}{4}} & e^{\frac{-j \times 2 \times \pi \times 1 \times 3}{4}} \\ e^{\frac{-j \times 2 \times \pi \times 2 \times 0}{4}} & e^{\frac{-j \times 2 \times \pi \times 2 \times 1}{4}} & e^{\frac{-j \times 2 \times \pi \times 2 \times 2}{4}} & e^{\frac{-j \times 2 \times \pi \times 2 \times 3}{4}} \\ e^{\frac{-j \times 2 \times \pi \times 3 \times 0}{4}} & e^{\frac{-j \times 2 \times \pi \times 3 \times 1}{4}} & e^{\frac{-j \times 2 \times \pi \times 3 \times 2}{4}} & e^{\frac{-j \times 2 \times \pi \times 3 \times 3}{4}} \end{bmatrix}$$

$$U = \begin{bmatrix} e^{\frac{-j2\pi 0x0}{4}} & e^{\frac{-j2\pi 0x1}{4}} & e^{\frac{-j2\pi 0x2}{4}} & e^{\frac{-j2\pi 0x3}{4}} \\ e^{\frac{-j2\pi 1x0}{4}} & e^{\frac{-j2\pi 1x1}{4}} & e^{\frac{-j2\pi 1x2}{4}} & e^{\frac{-j2\pi 1x3}{4}} \\ e^{\frac{-j2\pi 2x0}{4}} & e^{\frac{-j2\pi 2x1}{4}} & e^{\frac{-j2\pi 2x2}{4}} & e^{\frac{-j2\pi 2x3}{4}} \\ e^{\frac{-j2\pi 3x0}{4}} & e^{\frac{-j2\pi 3x1}{4}} & e^{\frac{-j2\pi 3x2}{4}} & e^{\frac{-j2\pi 3x3}{4}} \end{bmatrix}$$

$$e^{-j\varpi} = \cos \varpi - j \sin \varpi$$

$$\Rightarrow U = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

□ Tương tự cho ma trận V:

$$V(y, v) = e^{-\frac{j2\pi yv}{N}}; y, v = 0:N-1$$

$$V = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$G = U^* g^* V = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

$$G = \begin{bmatrix} 4 & -2-j2 & 0 & -2+j2 \\ -2-j2 & j2 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ -2+2j & 2 & 0 & -j2 \end{bmatrix}$$

= Real + j*Imp

Code

Sử dụng Lib `numpy` as `np`

```
import numpy as np

# Tạo ảnh gốc 4x4
g = np.array([[0, 0, 0, 0],
              [0, 1, 1, 0],
              [0, 1, 1, 0],
              [0, 0, 0, 0]])
```

```
# Tính DFT của ảnh g
G = np.fft.fft2(g)
```

```
# Hiển thị ảnh Fourier
print("Fourier:\n", G)
```



$$g(x,y) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$G = \begin{bmatrix} 4 & -2-j2 & 0 & -2+j2 \\ -2-j2 & j2 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ -2+j2 & 2 & 0 & -j2 \end{bmatrix}$$

Fourier:

```
[[ 4.+0.j -2.-2.j  0.+0.j -2.+2.j]
 [-2.-2.j  0.+2.j  0.+0.j  2.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [-2.+2.j  2.+0.j  0.+0.j  0.-2.j]]
```

Tìm Fourier của ảnh sau

$$g = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$



Fourier:

```
[[ 4.+0.j -4.+0.j  4.+0.j -4.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j  0.+0.j]]
```

Tìm Fourier của ảnh sau

$$g = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$



Fourier:

```
[[ 4.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [-4.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [ 4.+0.j  0.+0.j  0.+0.j  0.+0.j]
 [-4.+0.j  0.+0.j  0.+0.j  0.+0.j]]
```

Biến đổi Fourier trong xử lý ảnh

- ❑ Sau khi biến đổi ảnh Fourier sẽ có 2 thành phần thực và ảo. Một pixel sẽ có thể có thành phần
 - ❑ Phần thực
 - ❑ Hoặc Phần ảo
 - ❑ Hoặc cả phần thực và phần ảo

- ❑ Có thể biểu diễn DFT của 1 ảnh dưới hàm log:
$$d(u, v) = \log(1 + |G(u, v)|)$$

Biên độ và phổ

$|F(u, v)| = [R^2(u, v) + I^2(u, v)]^{1/2}$ được gọi là biên độ

$\phi(u) = \arctan \left[\frac{I(u, v)}{R(u, v)} \right]$ được gọi là phổ/góc pha

$P(u, v) = |F(u, v)|^2$ là năng lượng phổ

Tính chất

□ Liên hệ với tích chập

$$f(x, y) * g(x, y) \Leftrightarrow F(u, v)G(u, v)$$

$$f(x, y)g(x, y) \Leftrightarrow F(u, v) * G(u, v)$$